Two tree diseases, one causal core: oak powdery mildew and ash dieback

Simon Frost

Table of contents

1 Overview 1

2 A common shape for one season 2

3 Oak powdery mildew on regenerating oak 3
- 3.1 The best stock depends on the inoculum context . . . 5
- 3.2 The persistence branch . . . 6

4 Ash dieback and the litter reservoir 6
- 4.1 Litter removal, with and without the crown survey . . . 9
- 4.2 A healthy crown is not a clean bill of health . . . 9

5 What carries over, and what does not 10

References 10

1 Overview

Oak powdery mildew (the *Erysiphe* complex on *Quercus robur* and *Q. petraea*) and ash dieback (*Hymenoscyphus fraxineus* on *Fraxinus excelsior*) are both seasonal diseases of European trees whose epidemics carry over from one growing season to the next. They share a causal core: inoculum persisting from earlier seasons meets receptive host tissue, infection follows, and infection then feeds two branches, one replenishing the pathogen's inoculum for later seasons and one damaging the host (Marçais and Desprez-Loustau 2014; Combes et al. 2024).

They differ in where the inoculum comes from and in how infection becomes damage.

- Oak mildew is polycyclic: infected leaves produce conidia that infect more young leaves within the same season. Young leaves are susceptible and mature leaves resist, so the timing of leaf flushes against the timing of spring inoculum controls exposure, and an early first flush can escape (Hamelin et al. 2016; Dantec et al. 2015). Early flushing also

exposes the leaves to spring frost, and leaves replacing frost-killed ones flush late, into the epidemic (Marçais and Desprez-Loustau 2014; Dantec et al. 2015). Infection reduces growth and seedling survival (Barrès et al. 2024).
- Ash dieback is driven by sexual spores from fruiting bodies on infected leaf stalks in the litter, which persist for several years, so the reservoir has a memory longer than one season (Laubray et al. 2024). Colonisation of leaves and the woody damage that kills trees are controlled by different conditions, and crown symptoms say little about how much of the foliage is colonised or how much inoculum it will leave (Marçais et al. 2023).

This vignette builds both systems from the same shape of component: one growing season as an open influence diagram whose boundary carries the persistent inoculum and a host trait from one season to the next. Seasons compose in series into a multi-season problem. The same functions then answer each system's management question:

- for oak, which flushing phenology to plant, early or late, and how the answer depends on how much inoculum the site carries;
- for ash, whether to remove leaf litter after a survey of crown symptoms, and what a healthy-looking crown says about the infection underneath.

The tables are illustrative: values chosen to follow the directions reported in the literature, not estimates fitted to data. The fitted models of the systems (Hamelin et al. 2016; Fritsch et al. 2026) could supply them; the structure is the point here.

2 A common shape for one season

The shared causal core, with each disease's variables:

Stage	Oak powdery mildew	Ash dieback
Persistence (boundary in)	overwintered inoculum `Inoc_y`	litter reservoir `Reservoir_y`
Host trait (boundary, passed through)	flushing phenology `Flush`	tolerance `Tolerance`
Inoculum and exposure	frost `Frost_y`, exposure of young leaves `Exposure_y`	summer weather `Summer_y`, spores `Spores_y`
Infection	primary infection `Primary_y`, amplified to the season's burden `Burden_y`	foliar colonisation `Colonisation_y`
Persistence branch (boundary out)	next inoculum `Inoc_(y+1)`	next reservoir `Reservoir_(y+1)`, which keeps older litter
Damage branch	seedling performance `Performance_y`	woody damage `Woody_y`, seen as crown symptoms `Crown_y`

Each season is an open diagram from `(persistence, host trait)` to `(next persistence, host trait)`. Only variables cross the boundary; a season's decisions and utilities stay inside it, and its value reaches the rest of the chain through the semantics (ADR 0018).

A small helper builds conditional tables from a rule giving the child's distribution for each combination of parent states. A kernel's table lists the child first.

```
using CategoricalInfluenceDiagrams

LH = [:low, :high]
A(x, labels) = FiniteAxis(x, labels)
v(stem, y) = Symbol(stem, y)

function rule_table(child, labels, parents::Vector{FiniteAxis}, rule::Function)
    t = zeros(length(labels), length.(parents)...)
    for ci in CartesianIndices(Tuple(length.(parents)))
        states = ntuple(i -> parents[i].labels[ci[i]], length(parents))
        t[:, Tuple(ci)...] = rule(states...)
    end
    dom = isempty(parents) ? FiniteSpace() : FiniteSpace(parents)
    return FiniteKernel(dom, FiniteSpace(A(child, labels)), t)
end
nothing # hide
```

3 Oak powdery mildew on regenerating oak

The decision is made once, at planting: stock that flushes early or late. The flushing phenology is then a host trait that every season reads, observed by the manager.

```
FL = [:early, :late]; NY = [:no, :yes]
BURDEN = [:low, :moderate, :high]; PERF = [:good, :reduced]

planting = Open(influence_diagram(:Inoc1 ⇒ LH, :Flush ⇒ FL;
                                  decisions = [:Flush ⇒ Symbol[]], closed = false);
                inputs = [:Inoc1], outputs = [:Inoc1, :Flush], observed = [:Flush])

function oak_season(y; carry = true)
    vars = Any[v(:Inoc, y) ⇒ LH, :Flush ⇒ FL, v(:Frost, y) ⇒ NY,
               v(:Exposure, y) ⇒ [:escaped, :exposed], v(:Primary, y) ⇒ LH,
               v(:Burden, y) ⇒ BURDEN, v(:Performance, y) ⇒ PERF]
    mechanisms = Any[v(:Frost, y) ⇒ :Flush,
                     v(:Exposure, y) ⇒ (:Flush, v(:Frost, y)),
                     v(:Primary, y) ⇒ (v(:Inoc, y), v(:Exposure, y)),
                     v(:Burden, y) ⇒ v(:Primary, y),
                     v(:Performance, y) ⇒ (v(:Burden, y), v(:Frost, y))]
    if carry
        push!(vars, v(:Inoc, y + 1) ⇒ LH)
        push!(mechanisms, v(:Inoc, y + 1) ⇒ v(:Burden, y))
```

```
    end
    id = influence_diagram(vars...; mechanisms = mechanisms,
                           utilities = [v(:SeedlingValue, y) ⇒ v(:Performance, y)],
                           closed = false)
    return Open(id; inputs = [v(:Inoc, y), :Flush],
                outputs = carry ? [v(:Inoc, y + 1), :Flush] : Symbol[], observed =
[:Flush])
end
nothing # hide
```

The tables encode the directions in the literature:

- early flushing is more exposed to spring frost (0.3 against 0.1);
- an early flush that escapes frost mostly matures before the inoculum arrives (exposed with probability 0.2), a late flush mostly does not (0.8), and frost forces a late replacement flush, which is exposed (0.9);
- exposed leaves meeting abundant inoculum are mostly infected (0.9), against 0.2 when inoculum is scarce and 0.05 for escaped leaves;
- within the season, primary infection is amplified into the season's burden;
- a heavy burden leaves abundant inoculum for the next season;
- performance falls with the burden and is halved by frost damage; a seedling performing well is worth 10, one with reduced performance 2.

```
function oak_tables(y)
    ks = Dict{Symbol,FiniteKernel}(
        v(:Frost, y) ⇒ rule_table(v(:Frost, y), NY, [A(:Flush, FL)],
                                  f -> f === :early ? [0.7, 0.3] : [0.9, 0.1]),
        v(:Exposure, y) ⇒ rule_table(v(:Exposure, y), [:escaped, :exposed],
                                     [A(:Flush, FL), A(v(:Frost, y), NY)],
                                     (f, fr) -> fr === :yes ? [0.1, 0.9] :
                                                f === :early ? [0.8, 0.2] : [0.2,
0.8]),
        v(:Primary, y) ⇒ rule_table(v(:Primary, y), LH,
                                    [A(v(:Inoc, y), LH), A(v(:Exposure, y),
[:escaped, :exposed])],
                                    (i, e) -> e === :escaped ? [0.95, 0.05] :
                                              i === :high ? [0.1, 0.9] : [0.8,
0.2]),
        v(:Burden, y) ⇒ rule_table(v(:Burden, y), BURDEN, [A(v(:Primary, y), LH)],
                                   p -> p === :low ? [0.7, 0.25, 0.05] : [0.1, 0.4,
0.5]),
        v(:Performance, y) ⇒ rule_table(v(:Performance, y), PERF,
                                        [A(v(:Burden, y), BURDEN), A(v(:Frost, y),
NY)],
                                        (b, fr) -> begin
                                            g = (b === :low ? 0.9 : b === :moderate
? 0.6 : 0.2) *
                                                (fr === :yes ? 0.5 : 1.0)
                                            [g, 1 - g]
```

```
                                        end),
        v(:Inoc, y + 1) ⇒ rule_table(v(:Inoc, y + 1), LH, [A(v(:Burden, y),
        BURDEN)],
                                    b -> b === :low ? [0.9, 0.1] :
                                         b === :moderate ? [0.5, 0.5] : [0.1,
0.9]))
    us = Dict{Symbol,AbstractUtility}(
        v(:SeedlingValue, y) ⇒ TabularUtility([A(v(:Performance, y), PERF)], [10.0,
        2.0]))
    return ks, us
end

oak_kernels = Dict{Symbol,FiniteKernel}()
oak_utilities = Dict{Symbol,AbstractUtility}()
for y in 1:2
    ks, us = oak_tables(y)
    merge!(oak_kernels, ks)
    merge!(oak_utilities, us)
end
oak = compose(compose(planting, oak_season(1)), oak_season(2; carry = false))
nothing # hide
```

3.1 The best stock depends on the inoculum context

The planting decision is evaluated on the two-season composite, against a prior on the inoculum the site carries at planting. `contextual_optimize` searches the decision in that context:

```
contexts = ["scarce" ⇒ [1.0, 0.0], "mixed" ⇒ [0.5, 0.5], "abundant" ⇒ [0.0, 1.0]]
[(inoculum = name,
  best_stock = policy_table(contextual_optimize(oak, oak_kernels, oak_utilities;
                                                prior = prior).strategy[:Flush])[],
  early = contextual_value(oak, oak_kernels, oak_utilities,
                           Strategy(:Flush ⇒ ConstantPolicy(:early)); prior =
                           prior),
  late = contextual_value(oak, oak_kernels, oak_utilities,
                          Strategy(:Flush ⇒ ConstantPolicy(:late)); prior = prior))
 for (name, prior) in contexts]
```

```
3-element Vector{@NamedTuple{inoculum::String, best_stock::Symbol, early::Float64,
late::Float64}}:
 (inoculum = "scarce", best_stock = :late, early = 14.102581448000072, late =
 14.592214852799986)
 (inoculum = "mixed", best_stock = :early, early = 13.791967760000038, late =
 13.637460335999986)
 (inoculum = "abundant", best_stock = :early, early = 13.481354072000004, late =
 12.682705819199985)
```

Where inoculum is scarce, frost avoidance wins and late-flushing stock is better; where it is abundant, escape wins and early-flushing stock is better. This is the pattern found along elevation

gradients, where early flushing reduced mildew exposure at low elevation but not everywhere, and conflicted with frost avoidance (Dantec et al. 2015). A budburst–disease association is a statement about a context, not a fixed property of the stock.

3.2 The persistence branch

Infection feeds the next season's inoculum. The boundary law of the first season (planting and season 1 together) is the distribution of what crosses into season 2, here over the joint states of `Inoc2` and `Flush`:

```
first_season = compose(planting, oak_season(1))
for f in FL
    r = interpret(first_season, oak_kernels, oak_utilities,
                  Strategy(:Flush ⇒ ConstantPolicy(f)))
    law = boundary_law(r; prior = [0.5, 0.5])
    println(f, ": P(abundant inoculum next season) = ", round(sum(law[[2, 4]]);
    digits = 3))
end
```

```
early: P(abundant inoculum next season) = 0.347
late: P(abundant inoculum next season) = 0.431
```

Early flushing lowers the next season's inoculum as well as this season's infection: the decision acts on both branches after infection. The second season's value depends on it through the composite, which is why the decision is evaluated on the chain rather than one season at a time.

4 Ash dieback and the litter reservoir

Each season, ash trees of a given tolerance class meet spores produced from the litter reservoir. In autumn the manager surveys crown symptoms and decides whether to remove the fallen leaves, which reduces the reservoir carried into the next year. The reservoir keeps older litter, so a heavy year raises it beyond the following season.

```
SUMMER = [:moderate, :hot]; TOL = [:tolerant, :susceptible]
DAMAGE = [:none, :dieback]; CROWN = [:healthy, :symptomatic]; LITTER = [:leave,
:remove]

function ash_season(y; carry = true)
    vars = Any[v(:Reservoir, y) ⇒ LH, :Tolerance ⇒ TOL, v(:Summer, y) ⇒ SUMMER,
               v(:Spores, y) ⇒ LH, v(:Colonisation, y) ⇒ LH, v(:Woody, y) ⇒
               DAMAGE,
               v(:Crown, y) ⇒ CROWN, v(:Litter, y) ⇒ LITTER]
    mechanisms = Any[v(:Summer, y) ⇒ (),
                     v(:Spores, y) ⇒ (v(:Reservoir, y), v(:Summer, y)),
                     v(:Colonisation, y) ⇒ v(:Spores, y),
```

```
                    v(:Woody, y) ⇒ (v(:Colonisation, y), v(:Summer, y),
                    :Tolerance),
                    v(:Crown, y) ⇒ v(:Woody, y)]
    if carry
        push!(vars, v(:Reservoir, y + 1) ⇒ LH)
        push!(mechanisms,
              v(:Reservoir, y + 1) ⇒ (v(:Reservoir, y), v(:Colonisation, y),
              v(:Litter, y)))
    end
    id = influence_diagram(vars...; mechanisms = mechanisms,
                           decisions = [v(:Litter, y) ⇒ [v(:Crown, y)]],
                           utilities = [v(:DamageLoss, y) ⇒ v(:Woody, y),
                                        v(:LitterCost, y) ⇒ v(:Litter, y)],
                           closed = false)
    return Open(id; inputs = [v(:Reservoir, y), :Tolerance],
                outputs = carry ? [v(:Reservoir, y + 1), :Tolerance] : Symbol[])
end
nothing # hide
```

The tables:

- a hot summer is less likely (0.3) and reduces both the fruiting of the litter and the progression of damage, the two stages responding differently (Marçais et al. 2023);
- an abundant reservoir mostly produces abundant spores (0.85 in a moderate summer, 0.6 in a hot one), a scarce one rarely (0.2 and 0.1);
- spores drive foliar colonisation;
- woody damage needs colonisation and is less likely in hot summers and much less likely in tolerant trees, which tolerate colonisation rather than escape it (Combes et al. 2024);
- a crown survey sees woody damage imperfectly;
- the next reservoir rises with this season's colonisation and with the litter already there, and removing the fallen leaves cuts it to 30%;
- woody damage costs 30 and litter removal 0.5.

```
function ash_tables(y)
    ks = Dict{Symbol,FiniteKernel}(
        v(:Summer, y) ⇒ rule_table(v(:Summer, y), SUMMER, FiniteAxis[], () -> [0.7,
        0.3]),
        v(:Spores, y) ⇒ rule_table(v(:Spores, y), LH,
                                   [A(v(:Reservoir, y), LH), A(v(:Summer, y),
SUMMER)],
                                   (r, s) -> r === :low ?
                                             (s === :moderate ? [0.8, 0.2] : [0.9,
0.1]) :
                                             (s === :moderate ? [0.15, 0.85] :
[0.4, 0.6])),
        v(:Colonisation, y) ⇒ rule_table(v(:Colonisation, y), LH, [A(v(:Spores, y),
        LH)],
                                         s -> s === :low ? [0.8, 0.2] : [0.2,
0.8]),
```

```
        v(:Woody, y) ⇒ rule_table(v(:Woody, y), DAMAGE,
                                  [A(v(:Colonisation, y), LH), A(v(:Summer, y),
SUMMER),
                                   A(:Tolerance, TOL)],
                                  (c, s, t) -> begin
                                      p = (c === :high ? 0.6 : 0.05) *
                                          (s === :hot ? 0.4 : 1.0) *
                                          (t === :tolerant ? 0.3 : 1.0)
                                      [1 - p, p]
                                  end),
        v(:Crown, y) ⇒ rule_table(v(:Crown, y), CROWN, [A(v(:Woody, y), DAMAGE)],
                                  w -> w === :none ? [0.9, 0.1] : [0.2, 0.8]),
        v(:Reservoir, y + 1) ⇒ rule_table(v(:Reservoir, y + 1), LH,
                                          [A(v(:Reservoir, y), LH),
                                           A(v(:Colonisation, y), LH),
                                           A(v(:Litter, y), LITTER)],
                                          (r, c, l) -> begin
                                              p = min((c === :high ? 0.7 : 0.15) +
                                                      (r === :high ? 0.2 : 0.0),
0.95) *
                                                  (l === :remove ? 0.3 : 1.0)
                                              [1 - p, p]
                                          end))
    us = Dict{Symbol,AbstractUtility}(
        v(:DamageLoss, y) ⇒ TabularUtility([A(v(:Woody, y), DAMAGE)], [0.0,
        -30.0]),
        v(:LitterCost, y) ⇒ TabularUtility([A(v(:Litter, y), LITTER)], [0.0,
        -0.5]))
    return ks, us
end

ash_kernels = Dict{Symbol,FiniteKernel}()
ash_utilities = Dict{Symbol,AbstractUtility}()
for y in 1:2
    ks, us = ash_tables(y)
    merge!(ash_kernels, ks)
    merge!(ash_utilities, us)
end
ash = compose(ash_season(1), ash_season(2; carry = false); no_forgetting = false)
nothing # hide
```

The stand starts with an abundant reservoir with probability 0.6, and one tree in ten is in the tolerant class. The joint prior over `(Reservoir1, Tolerance)` lists the first variable fastest:

```
ash_prior = kron([0.1, 0.9], [0.4, 0.6])
nothing # hide
```

4.1 Litter removal, with and without the crown survey

The composite forgets: the second season's decision does not see the first's survey, so the exhaustive search solves it.

```
best = contextual_optimize(ash, ash_kernels, ash_utilities; prior = ash_prior)
(value = best.expected_utility,
 litter1 = policy_table(best.strategy[:Litter1]),
 litter2 = policy_table(best.strategy[:Litter2]))
```

```
(value = -14.255594241034956, litter1 = [:remove, :remove], litter2 = [:leave,
:leave])
```

The best strategy removes the litter after the first season whatever the crowns look like, and leaves it after the second, where nothing later is modelled to benefit. Three rules for the first season, side by side:

```
rule(first) = Strategy(:Litter1 ⇒ first, :Litter2 ⇒ ConstantPolicy(:leave))
crown_triggered = DeterministicPolicy(:Litter1, [A(:Crown1, CROWN)], A(:Litter1,
LITTER),
                                      [:leave, :remove])
[(rule = name,
  value = contextual_value(ash, ash_kernels, ash_utilities, rule(p); prior =
ash_prior))
 for (name, p) in ["never remove" ⇒ ConstantPolicy(:leave),
                   "remove where crowns are symptomatic" ⇒ crown_triggered,
                   "always remove" ⇒ ConstantPolicy(:remove)]]
```

```
3-element Vector{@NamedTuple{rule::String, value::Float64}}:
 (rule = "never remove", value = -15.600661803450002)
 (rule = "remove where crowns are symptomatic", value = -15.035942479176693)
 (rule = "always remove", value = -14.255594241035014)
```

Removing only where crowns are symptomatic recovers part of the gain; removing everywhere recovers all of it. The reason is in what the survey says.

4.2 A healthy crown is not a clean bill of health

The probability that a stand's foliage is heavily colonised, before the survey and given each survey result, is a ratio of evidence masses (ADR 0021). Here it is computed under the never-remove rule; the survey comes before the removal, so the rule does not change it.

```
never = rule(ConstantPolicy(:leave))
mass(ev) = evidence_mass(ash, ash_kernels, ash_utilities, never; prior = ash_prior,
                         evidence = ev)
prior_colonised = mass([:Colonisation1 ⇒ :high])
[(crown = c,
  probability_of_crown = mass([:Crown1 ⇒ c]),
```

```
  colonised_given_crown = mass([:Crown1 => c, :Colonisation1 => :high]) /
mass([:Crown1 => c]),
  colonised_before_survey = prior_colonised)
 for c in CROWN]
```

```
2-element Vector{@NamedTuple{crown::Symbol, probability_of_crown::Float64,
colonised_given_crown::Float64, colonised_before_survey::Float64}}:
 (crown = :healthy, probability_of_crown = 0.7155521699999994, colonised_given_crown
 = 0.41327753921842014, colonised_before_survey = 0.5198000000000007)
 (crown = :symptomatic, probability_of_crown = 0.28444783000000007,
 colonised_given_crown = 0.7877661081119866, colonised_before_survey =
 0.5198000000000007)
```

Most stands look healthy, and a healthy crown lowers the probability of heavy colonisation only modestly. Because colonisation feeds the reservoir whether or not the trees show damage, the litter beneath healthy-looking crowns carries much of next year's inoculum, and a rule triggered by symptoms leaves it there. Low crown damage is not low infection, nor a low contribution to future inoculum (Marçais et al. 2023).

5 What carries over, and what does not

The two systems were built from the same shape of season, composed in series, and analysed with the same functions. What differs is inside the seasons:

- oak's within-season amplification sits between primary infection and the season's burden, and its persistence resets each year through the burden alone;
- ash's reservoir keeps older litter, its damage branch is decoupled from its persistence branch, and its observation layer (the crown survey) sits downstream of damage, not of colonisation.

The illustrative tables would be replaced by the fitted models where they exist: the seasonal transmission model of oak mildew (Hamelin et al. 2016), the effects of mildew on seedling survival (Barrès et al. 2024), the persistence of the ash pathogen in litter (Laubray et al. 2024) and the mechanistic-statistical model of ash dieback's expansion, which also separates infection, symptoms and observation (Fritsch et al. 2026). Each would enter as the tables of one component, leaving the others unchanged.

References

Barrès, Benoit, Cyril Dutech, Gilles Saint-Jean, et al. 2024. "Demographic and Genetic Impacts of Powdery Mildew in a Young Oak (Quercus robur L.) Cohort." *Annals of Forest Science* 81: 44. https://doi.org/10.1186/s13595-024-01259-2.

Combes, Matt, Joan Webber, and Lynne Boddy. 2024. "Current Understanding and Future Prospects for Ash Dieback Disease with a Focus on Britain." *Forestry* 97 (5): 678–91. https://doi.org/10.1093/forestry/cpae040.

Dantec, Cécile Françoise, Hugo Ducasse, Xavier Capdevielle, Olivier Fabreguettes, Sylvain Delzon, and Marie-Laure Desprez-Loustau. 2015. "Escape of Spring Frost and Disease Through Phenological Variations in Oak Populations Along Elevation Gradients." *Journal of Ecology* 103 (4): 1044–56. https://doi.org/10.1111/1365-2745.12403.

Fritsch, Coralie, Marie Grosdidier, Anne Gégout-Petit, and Benoît Marçais. 2026. "Mechanistic-Statistical Model for the Expansion of Ash Dieback." *Theoretical Population Biology* 171: 1–21. https://doi.org/10.1016/j.tpb.2026.06.006.

Hamelin, Frédéric M., Anne Bisson, Marie-Laure Desprez-Loustau, Frédéric Fabre, and Ludovic Mailleret. 2016. "Temporal Niche Differentiation of Parasites Sharing the Same Plant Host: Oak Powdery Mildew as a Case Study." *Ecosphere* 7 (11): e01517. https://doi.org/10.1002/ecs2.1517.

Laubray, Simon, Marc Buée, and Benoît Marçais. 2024. "Hymenoscyphus fraxineus Persistence in the Ash Litter." *Plant Pathology* 73 (7): 1937–46. https://doi.org/10.1111/ppa.13948.

Marçais, Benoît, and Marie-Laure Desprez-Loustau. 2014. "European Oak Powdery Mildew: Impact on Trees, Effects of Environmental Factors, and Potential Effects of Climate Change." *Annals of Forest Science* 71 (6): 633–42. https://doi.org/10.1007/s13595-012-0252-x.

Marçais, Benoît, Arnaud Giraudel, and Claude Husson. 2023. "Ability of the Ash Dieback Pathogen to Reproduce and to Induce Damage on Its Host Are Controlled by Different Environmental Parameters." *PLOS Pathogens* 19 (4): e1010558. https://doi.org/10.1371/journal.ppat.1010558.